Gemini and CoPilot were both given the prompt below. Both GenAIs were accessed on a KU Linux machine using Firefox browser.

Prompt:

- “ Using recursive, object based implementation in C++, generate a program to find the solution to sudoku puzzles. The program will take a partially solved sudoku puzzle from a txt file where blank spaces are indicated by underscores. There are five files provided, named “puzzle1.txt”, “puzzle2.txt”, “puzzle3.txt”, “puzzle4.txt”, and “puzzle5.txt”. The sudoku puzzles are 9 by 9 grids divided into 3 by 3 subgrids. The program should use lists to store and iterate through a sudoku puzzle. There should be detailed and understandable comments explaining each code block, function, variable initialization, class initialization, or complex section.

  A puzzle should be solved with recursion through a depth-first search with backtracking; To solve a puzzle where k out of 81 cells are filled, the program should: Identify an empty cell and decide a list of values that could go in that empty cell. If none of the values could go in that empty cell (the values already appear in the same row, column, or subgrid), the result is a no-solution. Otherwise, if the first candidate value can go in the empty cell, place the first candidate value into the cell and move onto the next empty cell with a recursive function call. If the next empty cell returns a no-solution, we back track to the previous empty cell and use the next candidate value. This continues until the whole grid is filled successfully (k=81), or it has been determined that no solution exists (all candidate values return a no-solution).

  The output for each puzzle should print: the filename of the puzzle, the unsolved puzzle, the puzzle solution (or solutions if multiple solutions exist), and the number of solutions. If no solution is found, print “no solution found” instead of the puzzle solution.”

Gemini’s Program:

```
/**
* @file SudokuSolver.cpp
* @brief Object-based Sudoku Solver using Recursive Backtracking (DFS)
in C++.
*
* This program reads partially solved Sudoku puzzles from text files
where blank spaces
* are represented by underscores ('_'). It uses a class-based approach,
storing the
* grid in nested vectors (lists), and solves each puzzle using
depth-first search
* with backtracking. It outputs the filename, the unsolved puzzle, all
solutions
* found, and the total solution count.
*/
```

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <fstream>

/**
* @class SudokuPuzzle
* @brief Represents a 9x9 Sudoku puzzle board and provides methods to
load,
*        validate, and solve the puzzle using recursive backtracking.
*/
class SudokuPuzzle {
private:
   // 2D vector (list of lists) to store the 9x9 grid characters
('1'-'9' or '_')
   std::vector<std::vector<char>> grid;

public:
   /**
    * @brief Construct a new Sudoku Puzzle object.
    * Initializes a 9x9 grid filled with placeholder underscores ('_').
    */
   SudokuPuzzle() {
       grid.resize(9, std::vector<char>(9, '_'));
   }

   /**
    * @brief Loads a Sudoku puzzle from a text file.
    *
    * @param filename The path/name of the file to open.
    * @return true If the file was successfully opened and read.
    * @return false If the file could not be opened.
    */
   bool loadFromFile(const std::string& filename) {
       std::ifstream file(filename);
       if (!file.is_open()) {
           return false;
       }
```

```
        // Read 9 rows and 9 columns from the file
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                file >> grid[i][j];
            }
        }
        file.close();
        return true;
    }

    /**
     * @brief Prints the current state of the puzzle grid to standard
output.
     */
    void print() const {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                std::cout << grid[i][j] << " ";
            }
            std::cout << "\n";
        }
    }

    /**
     * @brief Validates if placing a specific character in a cell follows
Sudoku rules.
     *
     * Checks if the value already exists in the given row, column, or
3x3 subgrid.
     *
     * @param row The row index (0-8) of the cell.
     * @param col The column index (0-8) of the cell.
     * @param val The character value ('1'-'9') to check.
     * @return true If the value is valid according to Sudoku rules.
     * @return false If the value violates row, column, or subgrid
constraints.
     */
    bool isValid(int row, int col, char val) const {
```

```
        // 1. Check the row and column constraints
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == val) return false; // Found in row
            if (grid[i][col] == val) return false; // Found in column
        }

        // 2. Check the 3x3 subgrid constraint
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (grid[startRow + i][startCol + j] == val) {
                    return false; // Found in the 3x3 subgrid
                }
            }
        }

        return val; // Valid placement
    }

    /**
     * @brief Iterates through the grid to find the next empty cell.
     *
     * @param row Reference variable updated with the row index of the
empty cell.
     * @param col Reference variable updated with the column index of the
empty cell.
     * @return true If an empty cell ('_') is found.
     * @return false If the entire grid is filled (no empty cells
remain).
     */
    bool findEmptyCell(int& row, int& col) const {
        for (row = 0; row < 9; ++row) {
            for (col = 0; col < 9; ++col) {
                if (grid[row][col] == '_') {
                    return true; // Empty cell located
                }
            }
        }
```

```
        return false; // Puzzle is completely filled
    }

    /**
     * @brief Recursive Depth-First Search function with backtracking to
find all solutions.
     *
     * Identifies an empty cell, iterates through candidate values
('1'-'9'),
     * places valid candidates, and recurses to the next empty cell.
Backtracks
     * by resetting cells if a branch leads to a dead end.
     *
     * @param solutionCount Reference counter tracking the number of
valid solutions found.
     * @param solutions Reference container storing all valid solution
grids.
     */
    void solveAll(int& solutionCount,
std::vector<std::vector<std::vector<char>>>& solutions) {
        int row, col;

        // Base case: If no empty cell remains, a complete valid solution
is achieved (k=81)
        if (!findEmptyCell(row, col)) {
            solutionCount++;
            solutions.push_back(grid);
            return;
        }

        // Try candidate values from '1' to '9' for the current empty
cell
        for (char val = '1'; val <= '9'; ++val) {
            if (isValid(row, col, val)) {
                // Place candidate value into the cell
                grid[row][col] = val;

                // Move onto the next empty cell with a recursive
function call
```

```
                solveAll(solutionCount, solutions);

                // Backtrack: Reset cell to empty if the recursive branch
fails
                grid[row][col] = '_';
            }
        }
    }

    /**
     * @brief Helper function to process, solve, and print the results
for a specific puzzle file.
     *
     * @param filename The file name containing the puzzle.
     */
    static void processPuzzle(const std::string& filename) {
        std::cout << "========================================\n";
        std::cout << "Puzzle Filename: " << filename << "\n";
        std::cout << "========================================\n";

        SudokuPuzzle puzzle;

        // Attempt to load the puzzle from the file
        if (!puzzle.loadFromFile(filename)) {
            std::cout << "Error: Could not open file " << filename <<
"\n\n";
            return;
        }

        // Print the initial unsolved puzzle
        std::cout << "Unsolved Puzzle:\n";
        puzzle.print();
        std::cout << "\n";

        // Containers to store solutions and solution counts
        int solutionCount = 0;
        std::vector<std::vector<std::vector<char>>> solutions;

        // Execute backtracking solver
```

```
        puzzle.solveAll(solutionCount, solutions);

        // Print number of solutions
        std::cout << "Number of Solutions: " << solutionCount << "\n\n";

        // Print puzzle solution(s) or "no solution found" message
        if (solutionCount == 0) {
            std::cout << "no solution found\n\n";
        } else {
            for (size_t s = 0; s < solutions.size(); ++s) {
                std::cout << "Solution " << (s + 1) << ":\n";
                for (int i = 0; i < 9; ++i) {
                    for (int j = 0; j < 9; ++j) {
                        std::cout << solutions[s][i][j] << " ";
                    }
                    std::cout << "\n";
                }
                std::cout << "\n";
            }
        }
    }
};

/**
* @brief Main execution function.
*
* Iterates through the five required puzzle files and processes each one
sequentially.
*/
int main() {
    // Array containing the names of the five target text files
    std::string puzzleFiles[] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };
```

```
    // Process each puzzle file sequentially
    for (const std::string& filename : puzzleFiles) {
        SudokuPuzzle::processPuzzle(filename);
    }

    return 0;
}
```

Copilot's Program:

```
/*****************************************************************
* Author: Student
* Program: Sudoku Solver
*
* Description:
* Reads Sudoku puzzles from text files and solves them
* using recursive depth-first search with backtracking.
*
* Empty cells are represented with underscores (_).
*
* The solver finds all valid solutions.
*****************************************************************/

#include <iostream>
#include <fstream>
#include <list>
#include <vector>
#include <string>

using namespace std;

/*****************************************************************
* Sudoku Class
*
* Stores and solves a Sudoku puzzle.
*****************************************************************/
class Sudoku
{
private:
```

```
    /*
     * The board is stored as a list of rows.
     * Each row is itself a list of integers.
     *
     * Value 0 represents an empty cell.
     */
    list<list<int>> board;

    /*
     * Stores every valid solution found.
     */
    vector<list<list<int>>> solutions;

public:

    /*****************************************************************
    * Constructor
    *****************************************************************/
    Sudoku()
    {
    }

    /*****************************************************************
    * Load puzzle from file
    *
    * Returns true if file was loaded successfully.
    *****************************************************************/
    bool loadPuzzle(const string& filename)
    {
        board.clear();

        ifstream fin(filename);

        if (!fin)
        {
            return false;
        }
```

```
    char value;

    /*
     * Read 9 rows.
     */
    for (int r = 0; r < 9; r++)
    {
        list<int> row;

        /*
         * Read 9 entries.
         */
        for (int c = 0; c < 9; c++)
        {
            fin >> value;

            if (value == '_')
            {
                row.push_back(0);
            }
            else
            {
                row.push_back(value - '0');
            }
        }

        board.push_back(row);
    }

    fin.close();
    return true;
}

/*****************************************************************
* Print puzzle board.
*****************************************************************/
void printBoard(const list<list<int>>& puzzle) const
{
    int rowNum = 0;
```

```
    for (auto row = puzzle.begin();
         row != puzzle.end();
         ++row, ++rowNum)
    {
        int colNum = 0;

        for (auto col = row->begin();
             col != row->end();
             ++col, ++colNum)
        {
            if (*col == 0)
                cout << "_ ";
            else
                cout << *col << " ";

            if ((colNum + 1) % 3 == 0 &&
                colNum != 8)
            {
                cout << "| ";
            }
        }

        cout << endl;

        if ((rowNum + 1) % 3 == 0 &&
            rowNum != 8)
        {
            cout << "------+-------+------"
                 << endl;
        }
    }
}

/*****************************************************************
* Convert list board to vector board
*
* Using vectors internally simplifies indexing.
*****************************************************************/
```

```
vector<vector<int>> listToVector(
    const list<list<int>>& puzzle) const
{
    vector<vector<int>> grid;

    for (auto row : puzzle)
    {
        vector<int> currentRow;

        for (auto value : row)
        {
            currentRow.push_back(value);
        }

        grid.push_back(currentRow);
    }

    return grid;
}

/*****************************************************************
* Convert vector board back to list board.
*****************************************************************/
list<list<int>> vectorToList(
    const vector<vector<int>>& grid) const
{
    list<list<int>> puzzle;

    for (int r = 0; r < 9; r++)
    {
        list<int> row;

        for (int c = 0; c < 9; c++)
        {
            row.push_back(grid[r][c]);
        }

        puzzle.push_back(row);
    }
```

```
    return puzzle;
}

/*****************************************************************
* Determines whether a number may be placed
* in a specified cell.
*****************************************************************/
bool isValid(vector<vector<int>>& grid,
             int row,
             int col,
             int value)
{
    /*
     * Check row.
     */
    for (int c = 0; c < 9; c++)
    {
        if (grid[row][c] == value)
        {
            return false;
        }
    }

    /*
     * Check column.
     */
    for (int r = 0; r < 9; r++)
    {
        if (grid[r][col] == value)
        {
            return false;
        }
    }

    /*
     * Determine top-left corner
     * of the 3x3 subgrid.
     */
```

```
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    /*
     * Check 3x3 box.
     */
    for (int r = startRow;
         r < startRow + 3;
         r++)
    {
        for (int c = startCol;
             c < startCol + 3;
             c++)
        {
            if (grid[r][c] == value)
            {
                return false;
            }
        }
    }

    return true;
}

/*****************************************************************
* Find the next empty cell.
*
* Returns true if found.
*****************************************************************/
bool findEmptyCell(vector<vector<int>>& grid,
                   int& row,
                   int& col)
{
    for (row = 0; row < 9; row++)
    {
        for (col = 0; col < 9; col++)
        {
            if (grid[row][col] == 0)
            {
```

```
                return true;
            }
        }
    }

    return false;
}

/*****************************************************************
* Recursive DFS Backtracking Solver
*
* Algorithm:
* 1. Find an empty cell.
* 2. Generate candidate values 1-9.
* 3. Test each candidate.
* 4. Recursively solve remaining puzzle.
* 5. Backtrack if necessary.
*****************************************************************/
void solveRecursive(vector<vector<int>>& grid)
{
    int row;
    int col;

    /*
     * Base Case:
     * No empty cells remain.
     */
    if (!findEmptyCell(grid, row, col))
    {
        solutions.push_back(
            vectorToList(grid));

        return;
    }

    /*
     * Try candidate values 1-9.
     */
    for (int candidate = 1;
```

```
             candidate <= 9;
             candidate++)
        {
            if (isValid(grid,
                        row,
                        col,
                        candidate))
            {
                /*
                 * Make choice.
                 */
                grid[row][col] = candidate;

                /*
                 * Explore.
                 */
                solveRecursive(grid);

                /*
                 * Undo choice.
                 * (BACKTRACK)
                 */
                grid[row][col] = 0;
            }
        }
    }

    /**************************************************************
    * Solve puzzle and collect all solutions.
    **************************************************************/
    void solve()
    {
        solutions.clear();

        vector<vector<int>> grid =
            listToVector(board);

        solveRecursive(grid);
    }
```

```
/*****************************************************************
* Display all results.
*****************************************************************/
void printResults(const string& filename)
{
    cout << "\n==================================\n";
    cout << "Puzzle File: "
         << filename
         << endl;

    cout << "\nUnsolved Puzzle:\n\n";
    printBoard(board);

    cout << "\n";

    if (solutions.empty())
    {
        cout << "No solution found\n";
        cout << "Number of Solutions: 0\n";
        return;
    }

    int count = 1;

    for (auto& solution : solutions)
    {
        cout << "\nSolution "
             << count
             << ":\n\n";

        printBoard(solution);
        cout << endl;

        count++;
    }

    cout << "Number of Solutions: "
         << solutions.size()
```

```
            << endl;
    }
};

/*****************************************************************
* Main Function
*
* Processes all 5 puzzle files.
*****************************************************************/
int main()
{
    vector<string> files =
    {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const string& filename : files)
    {
        Sudoku puzzle;

        if (!puzzle.loadPuzzle(filename))
        {
            cout << "\nUnable to open file: "
                 << filename
                 << endl;
            continue;
        }

        puzzle.solve();
        puzzle.printResults(filename);
    }

    return 0;
}
```

Analysis:

1. Gemini
    a. Correctness:
        i. When given expected input: There is correct output for all cases. When there are 0, 1 or 1+ solutions, the unsolved puzzle, solved puzzle(s), or no solution were all displayed properly. For example, puzzle5.txt has two solutions, and both are displayed.
        ii. When given unexpected input(file does not exist, dimensions are not 9x9, sudoku grid with no empty spaces, not enough filled spaces to solve grid/fully empty grid): When a file does not exist, the program will display an error message and continue to the next file(doesn't crash, handled properly). When the dimensions are not what is expected (ex: 9x8), it treats the missing row/col as one consisting of only empty cells, and attempts to find a solution. When the sudoku grid provided does not have any empty spaces (fully solved), it states there is one solution, which is technically correct. When the sudoku grid provided is empty, the program makes a large amount of recursive calls that basically freezes the program.
        iii. Other: –
    b. Execution Time [O(n)]:
        i. The program uses recursion to solve each sudoku puzzle. The amount of recursive calls depends on the number of unfilled sudoku squares (could range from 0 to 64 empty cells, so the worst case is 64 recursions or more if there are multiple solutions). O(n) for solveAll() because it recurses linearly.
    c. Space Complexity [O(1)]:
        i. The size of nested vectors is predetermined by requirements (9x9).
        ii. The space complexity is therefore O(1) because the size of vectors is constant. All variables initialized (iterators, storing values) are necessary and used efficiently.
    d. Maintainability:
        i. The program does not use namespace std, which prevents possible issues related to other namespace uses in other files if the program is integrated with other files in the future.
        ii. The program starts its iterators at 0 but preincrements instead of post (++i);this works identically to postincrement, but may conflict with code uniformity in comparison in other files(if integrated with other files in the future).
        iii. The program has comments explaining both large code blocks and individual lines.
        iv. There is a size_t type iterator in the loop used for printing all possible solutions; this accounts for the possibility of a large number of solutions.
2. CoPilot
    a. Correctness:

i. When given expected input: There is correct output for all cases. When there are 0, 1 or 1+ solutions, the unsolved puzzle, solved puzzle(s), or no solution were all displayed properly. For example, puzzle4.txt has no solutions, and the correct “no solutions” message is printed.
ii. When given unexpected input(file does not exist, dimensions are not 9x9,sudoku grid with no empty spaces): When a file does not exist, the program will display an error message and continue to the next file(doesn't crash, handled properly). When the dimensions are not what is expected (ex: 9x8), it treats the missing row/col as one consisting of only empty cells, and attempts to find a solution. When the sudoku grid provided does not have any empty spaces (fully solved), it states there is one solution, which is technically correct.
iii. Other: The program uses c++ lists, which is not ideal. The written prompt could have been better worded in specifying data type used to store sudoku puzzles instead of just using the generic term “list”. Because of this, there are unnecessary functions that convert lists to vectors and vectors to lists.

b. Execution Time [O(n)]:
i. Functions that convert list to vector and then back to list add unnecessary runtime.
ii. The program uses recursion to solve each sudoku puzzle. The amount of recursive calls depends on the number of unfilled sudoku squares (could range from 0 to 64 empty cells, so the worst case is 64 recursions or more if there are multiple solutions).

c. Space Complexity [O(1)]:
i. Using <list> and <vector> uses more memory compared to just using <vector>.
ii. The size of nested vectors is predetermined by requirements (9x9).
iii. The space complexity is therefore O(1) because the size of vectors is constant.

d. Maintainability:
i. The program uses auto for some variable inputs, which can affect readability/understanding of variable types for others reading the code.
ii. The program also uses namespace std, which could cause issues depending on possible integration with other files in the future.
iii. Mostly only has comments summarizing large blocks, inner workings of blocks are mostly uncommented or have very simple comments; variables are often initialized without explanation
iv. The use of lists and vectors interchangeably may cause confusion.
v. The program’s output is more readable as it uses bars and dashes to divide the sudoku grid.

Program Selected: Gemini

1. Justification

    a. Gemini's implementation makes more sense from a logical standpoint. Its object oriented approach is well organized; the SudokuPuzzle class and its functions handle all parts of the program efficiently. Comments are more abundant and well written. Unexpected inputs are handled favorably, and the use of vectors is efficient and convenient.
2. Changes Made
    a. Added personal comments to summarize code in own words/ showcase personal understanding of code/ further clarify specific parts of code.
    b. Added detection for at least 17 filled cells.